Reporte: Actividad 3

“Diseño de Backend con reglas de negocio y control de estados”

Alumna: Jocelyn Flores Gutiérrez

Maestro: Edgar Ivan Patricio Aizpuru

Materia: Desarrollo Full Stack

Fecha: 07/02/2026

● Índice

● Índice..... 2
● **Entidad principal**..... **3**
● **Reglas de negocio**..... **4**
● **Estructura del proyecto**..... **5**
● **Endpoints**..... **6**
● **Pruebas de funcionamiento**..... **7**

Diseñar e implementar un backend con Express y POO *que no solo haga CRUD, sino que aplique reglas de negocio reales,* ***separando correctamente responsabilidades*** *entre: routes, controllers, repositories*

El énfasis de esta actividad NO está en la cantidad de endpoints, sino en cómo se piensa y estructura la lógica usando los conceptos vistos.

Repositorio en Github:
https://github.com/JoFloresGutierrez06/Actividad-3-Full-Stack

Documentación de Postman:
https://www.postman.com/jofloresgutierrez-602355/workspace/actividad-3-full-stack

● Entidad principal

Entidad: Pedido

- **id** (Number - int) - Privado
- **producto** (String) - Privado
- **cantidad** (Number - int) - Privado
- **estado** (String) - Privado

Estado: “pendiente” - “Confirmado” - Cancelado”

En este caso se decidió que todos los atributos sean privados para seguir los principios de encapsulamiento, evitando que se manipulen los datos directamente (por motivos de seguridad) y, en su lugar, se haga a través de métodos, como getters y setters.

Implementaciones futuras:

- El atributo “producto” podría cambiar para tratarse de varios productos, con un arreglo de strings o json que permita agregar más características del producto (como su id, categoría o número de serie).
    - Ejemplo 1: [“Laptop”, “Mouse”, “Monitor”, “Cereal”]
    - Ejemplo 2: [{nombre: “Laptop”, numSerie: 1234},
      {nombre: “Mouse”, numSerie: 4567}]
    - Nota: eso sería considerando que un único pedido puede manejar varios artículos, pero si se compra individual con varios proveedores (como pienso que es el caso de esta actividad), es lógico que se maneje un único artículo.
- Se puede añadir el atributo “precio” al producto para luego realizar una operación en relación a la cantidad del producto y el porcentaje de comisión de la empresa para realizar el pedido, devolviendo un “costo final” del envío para el usuario.

● Reglas de negocio

Generales:

- **Id:** Tiene un autoincremento que no puede ser intervenido.
- **Cantidad:** Debe ser mayor a 0.
- **Estado:** Únicamente admite los estados “Pendiente”, “Confirmado” y “Cancelado”

Creación de pedido:

- **Cantidad:** Debe ser mayor a 0.
- **Estado:** Un pedido *nuevo* siempre inicia en estado “Pendiente”

Cambio de estado:

- Un pedido “*Pendiente*” puede pasar a “Confirmado” o “Cancelado”.
- Un pedido “*Confirmado”* NO puede modificarse
- Un pedido “*Cancelado”* NO puede modificarse
- (400 + mensaje claro): Si se intenta una acción inválida

Eliminación:

- Solo se pueden eliminar pedidos en estado "pendiente"
- (400): Si se intenta eliminar un pedido "confirmado" o "cancelado"

Estas reglas de negocio se establecen principalmente en el archivo de repositorio, especialmente toda la parte de lógica para guardar los datos o crear los pedidos, sin embargo, la parte que valida que los datos estén correctos se encuentra en el controlador, desde ahí confirma que los datos que serán pasados al repositorio son del tipo de dato correcto, que no se encuentran vacíos y que tienen un valor mayor a 0, etc. En caso de incumplir alguno de los requisitos devuelve el tipo de error correspondiente para el HTTP.

- Se encuentran disponibles en el archivo: *pedidos.repository.js* y *pedidos.controller.js*

Se busca esa separación en los archivos por temas de practicidad y organización, pues al saber dónde se encuentran ubicadas cada una de las partes resulta más sencillo identificar y corregir errores, así como realizar cambios en caso de ser necesario (como la modificación de alguna de las reglas de negocio o la adición de un nuevo atributo o petición HTTP).

● Estructura del proyecto

/src
- **/routes**
  - *pedidos.routes.js*
- **/controllers**
  - *pedidos.controller.js*
- **/repositories**
  - *pedidos.repository.js*

index.js

- Almacenamiento en memoria - Sin base de datos

Repository:

- Contiene una clase “PedidosRepository”.
- Guarda/lee los datos

Controller:

- Valida los datos
- Valida las reglas de negocio
- Decide los códigos HTTP (status)

Rutas:

- Redirige las peticiones HTTP al controlador

• Endpoints

- **GET /pedidos** **-** Muestra todos los pedidos
- **GET /pedidos/:id** - Muestra un pedido específico a través de su id
- **POST /pedidos** - Permite registrar un nuevo pedido
- **PUT /pedidos/:id** - Permite modificar un pedido previamente registrado
- **DELETE /pedidos/:id** - Permite eliminar únicamente los pedidos cancelados

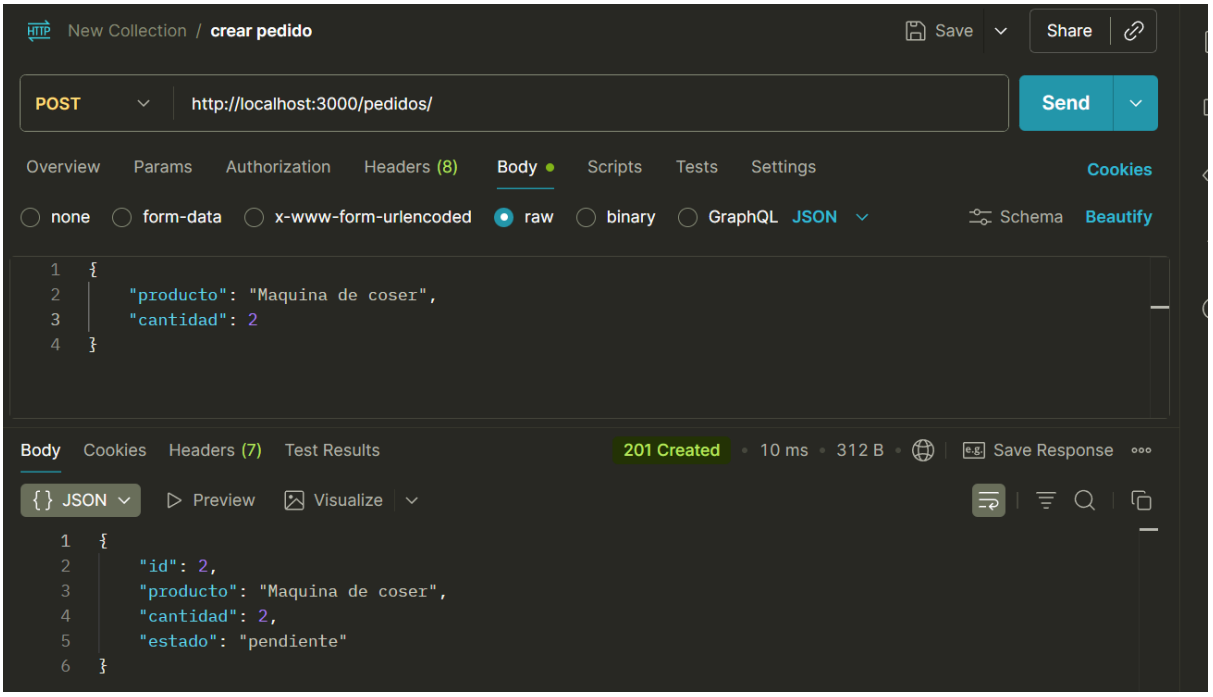
---

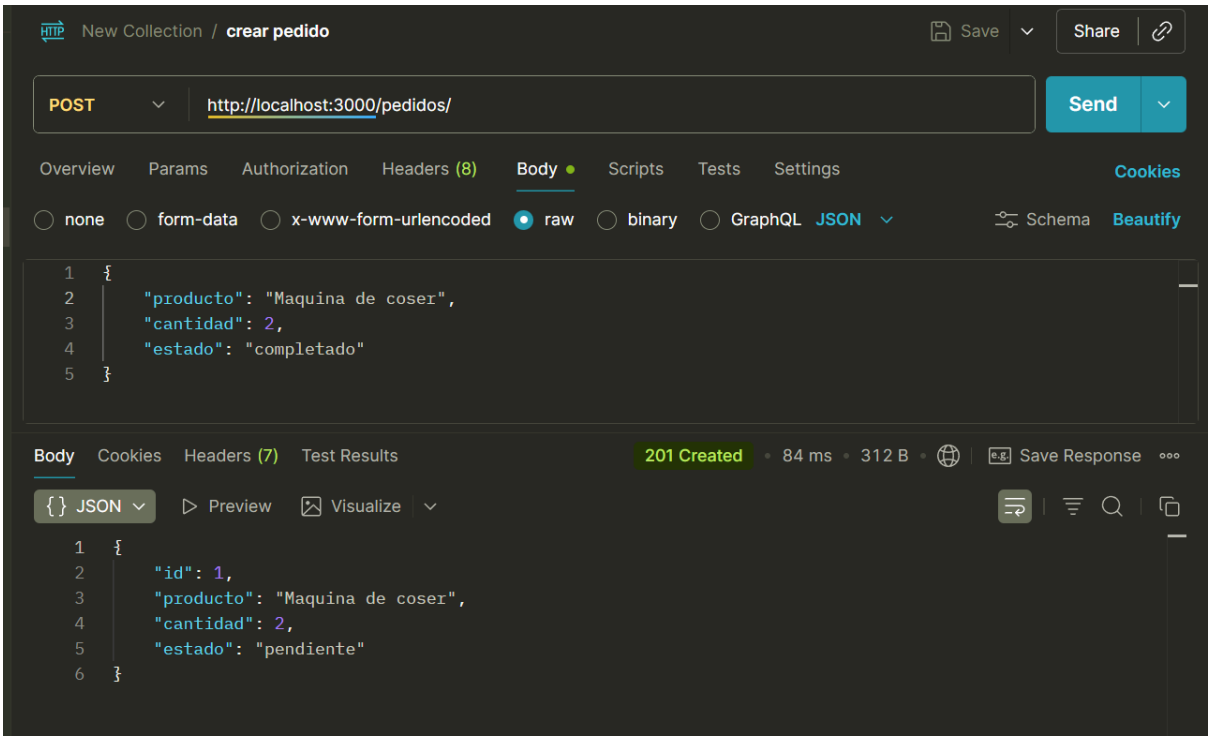
También se pueden añadir los siguientes endpoints:

- **GET /pedidos/pendientes** **-** Muestra los pedidos con estatus “pendiente”
- **GET /pedidos/cancelados** **-** Muestra los pedidos con estatus “cancelado”
- **GET /pedidos/confirmados** **-** Muestra los pedidos con estatus “confirmado”

• Pruebas de funcionamiento

POST - crear pedido





Al momento de crear un pedido e intentar especificar un estado distinto a “pendiente” lo ignora.

GET - obtener pedidos (todos o por id)

New Collection / **get pedidos**

Save | Share

GET http://localhost:3000/pedidos/ Send

Overview Params Authorization Headers (6) Body Scripts Tests Settings Cookies

Body 200 OK • 7 ms • 523 B Save Response

{} JSON Preview Visualize


```
[
    {
        "id": 2,
        "producto": "Maquina de coser",
        "cantidad": 2,
        "estado": "cancelado"
    },
    {
        "id": 3,
        "producto": "Refrigerador Samsung",
        "cantidad": 3,
        "estado": "completado"
    },
    {
        "id": 4,
        "producto": "Refrigerador HP",
        "cantidad": 4,
        "estado": "pendiente"
    },
```


New Collection / **get pedido por id**

Save | Share

GET http://localhost:3000/pedidos/1 Send

Overview Params Authorization Headers (6) Body Scripts Tests Settings Cookies

get pedido por id View complete documentation →

Body Cookies Headers (7) Test Results 200 OK • 8 ms • 307 B Save Response

{} JSON Preview Visualize


```
{
    "id": 1,
    "producto": "Maquina de coser",
    "cantidad": 2,
    "estado": "pendiente"
}
```


New Collection / **get pedido por id**

Save | Share

GET http://localhost:3000/pedidos/9 Send

Overview Params Authorization Headers (6) Body Scripts Tests Settings Cookies

get pedido por id View complete documentation →

Body Cookies Headers (7) Test Results 404 Not Found • 7 ms • 274 B Save Response

{} JSON Preview Debug with AI


```
{
    "error": "Pedido no encontrado"
}
```

POST - crear pedidos

POST http://localhost:3000/pedidos/ Try

Params Headers Body

Query Params

Key	Value	Description	Bulk Edit

Body Headers (7) Status Code 201 Created

Raw Preview

```
{
    "id": 1,
    "producto": "Maquina de coser",
    "cantidad": 2,
    "estado": "pendiente"
}
```

New Collection / crear pedido Save Share

POST http://localhost:3000/pedidos/ Send

Overview Params Authorization Headers (8) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{
    "producto": "Libro"
}
```

Body 400 Bad Request 7 ms 274 B Save Response

JSON Preview Debug with AI

```
{
    "error": "Cantidad inválida"
}
```

POST http://localhost:3000/pedidos/ Try

Params Headers Body

Query Params

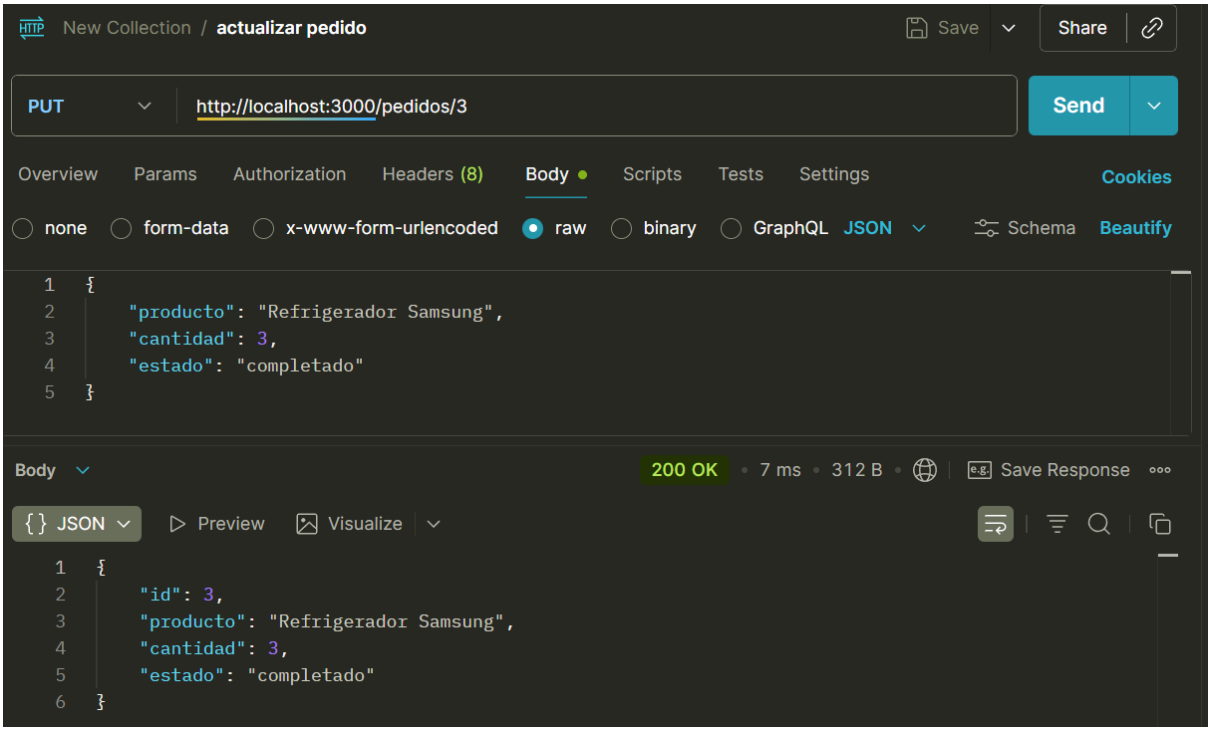
Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Headers (7) Status Code 400 Bad Request

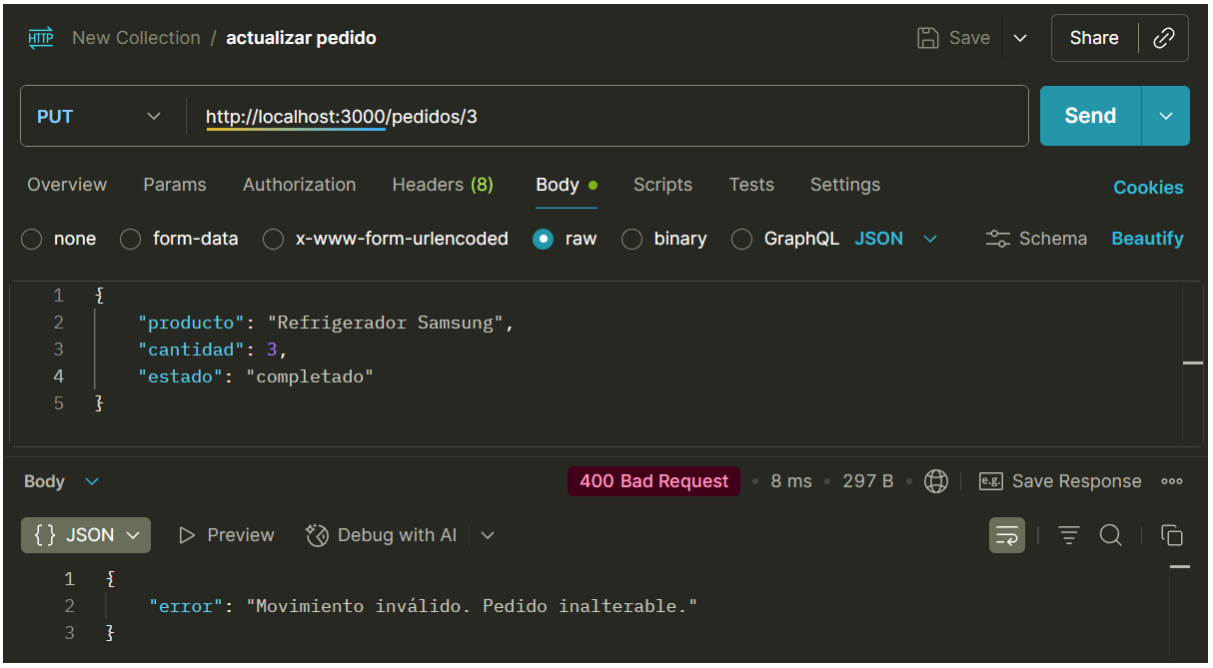
Raw Preview

```
{
    "error": "Cantidad inválida"
}
```

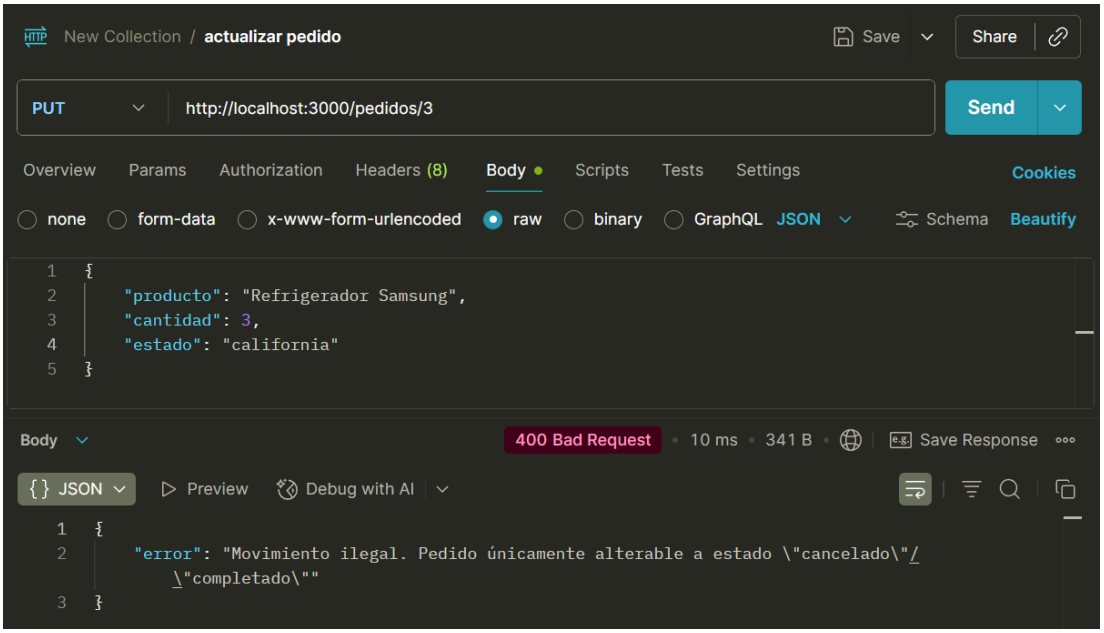
PUT - actualizar pedidos



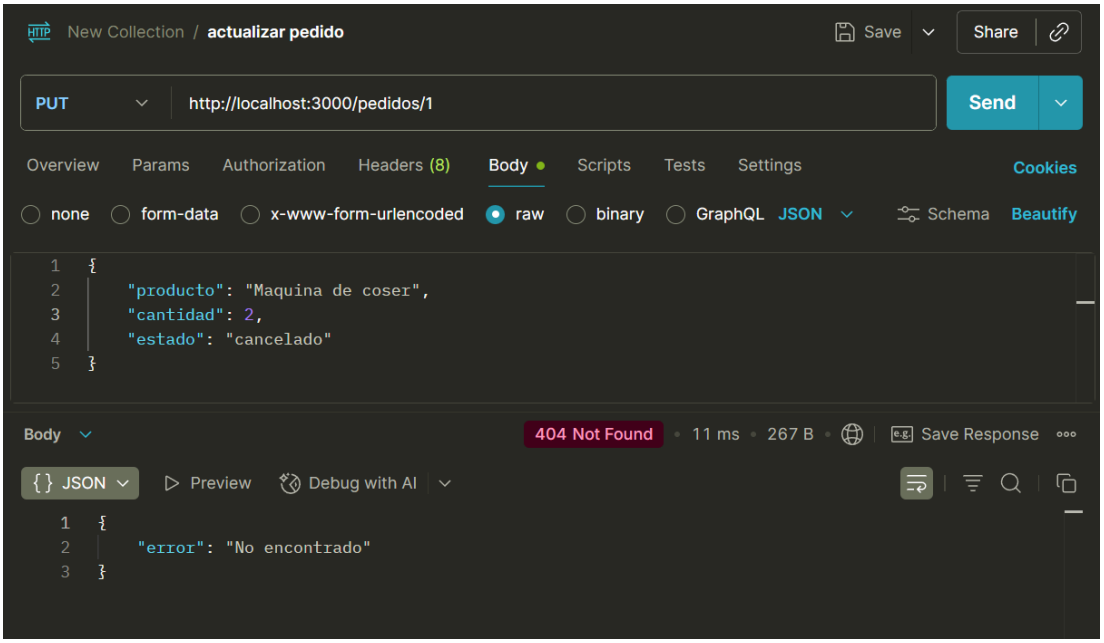
Actualizó un pedido en estado “pendiente”.

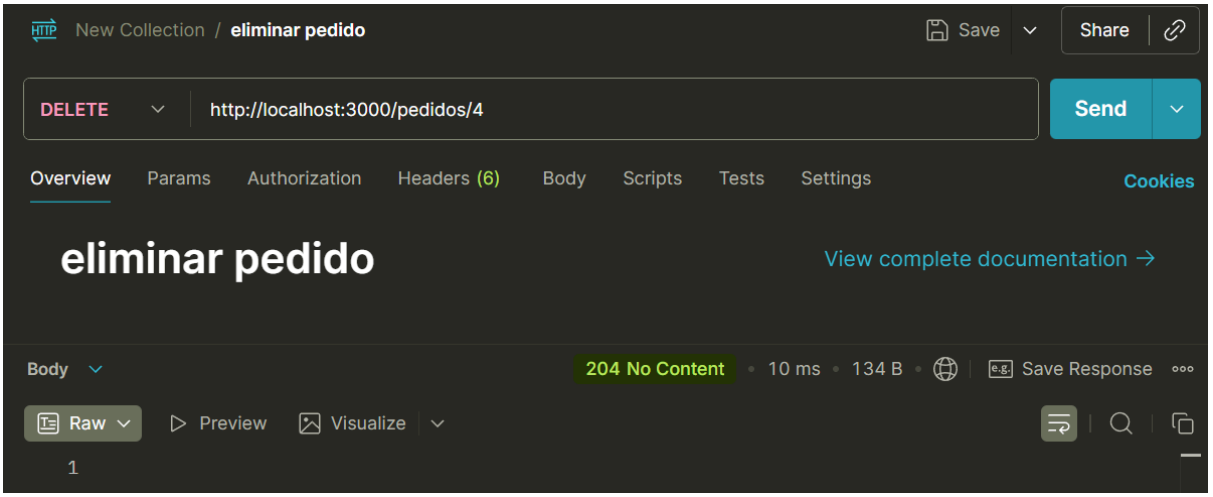
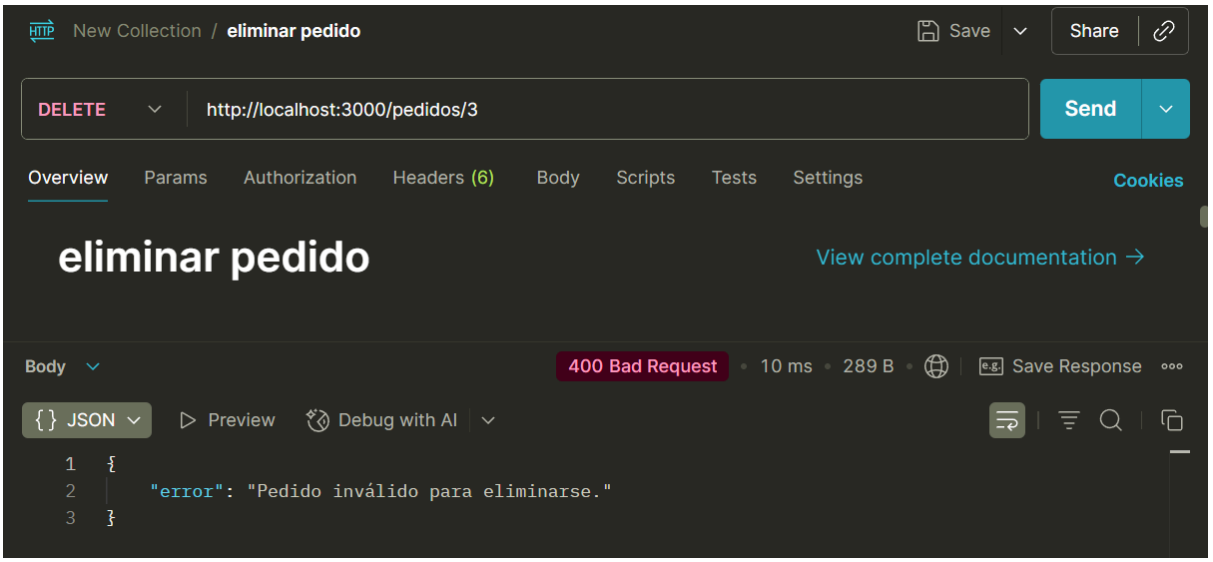


Intentar volver a alterarlo.



Intentar cambiar el estado a una opción fuera de “completado” o “cancelado”.



Intento de actualizar un pedido inexistente. (previamente fue eliminado)

DELETE - eliminar pedidos pendientes